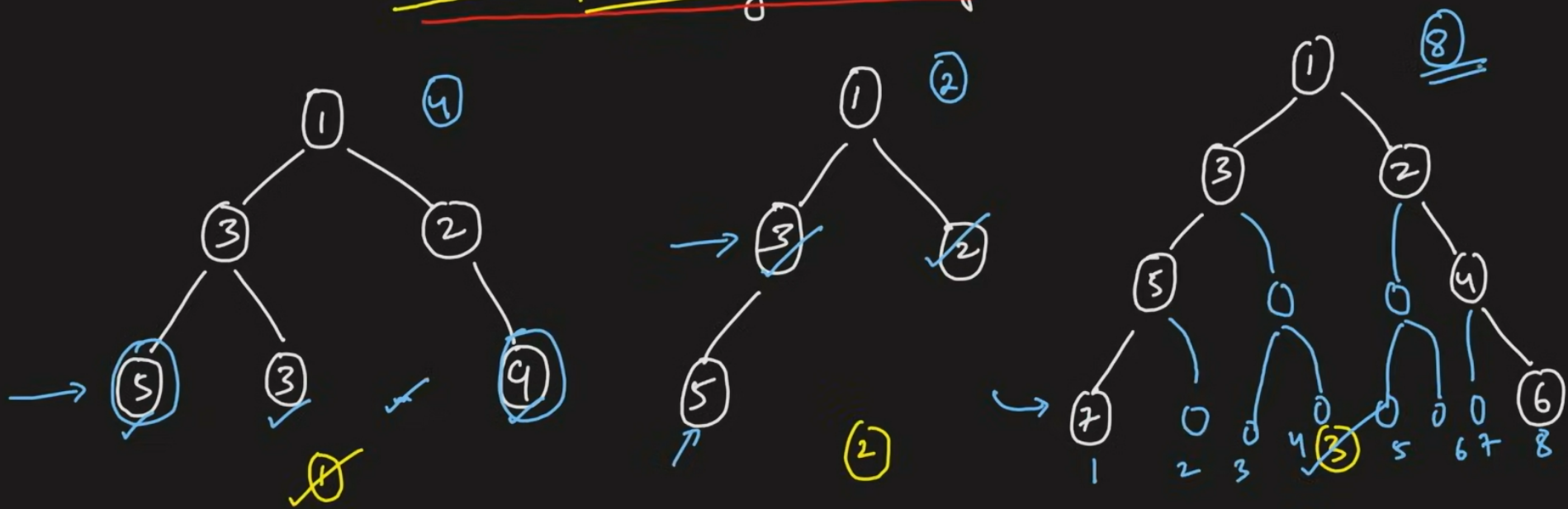
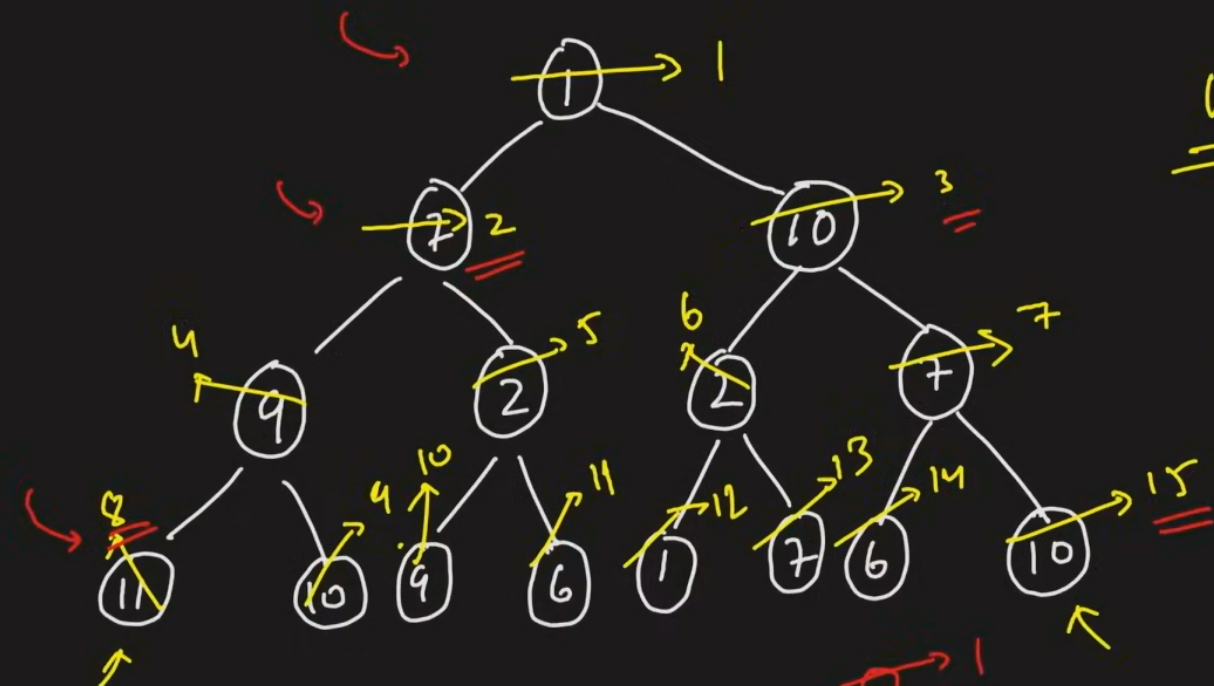


Maximum Width of a Binary Tree



Width $\rightarrow$ no of nodes ^{in a level} betw any 2 nodes

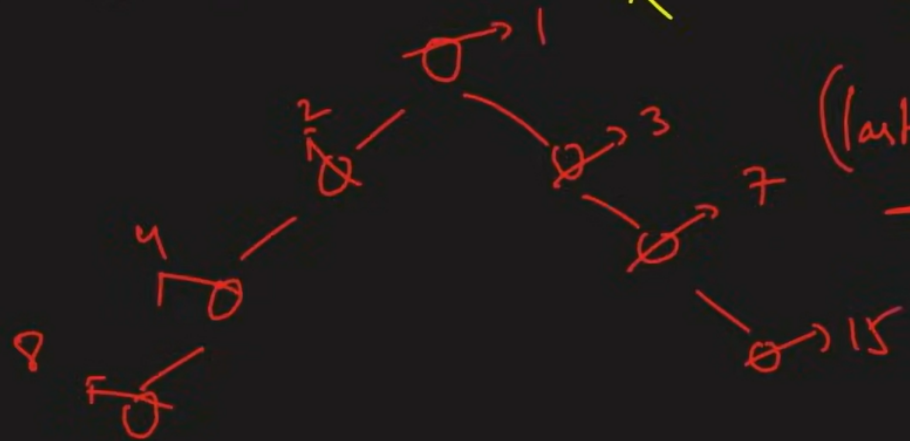


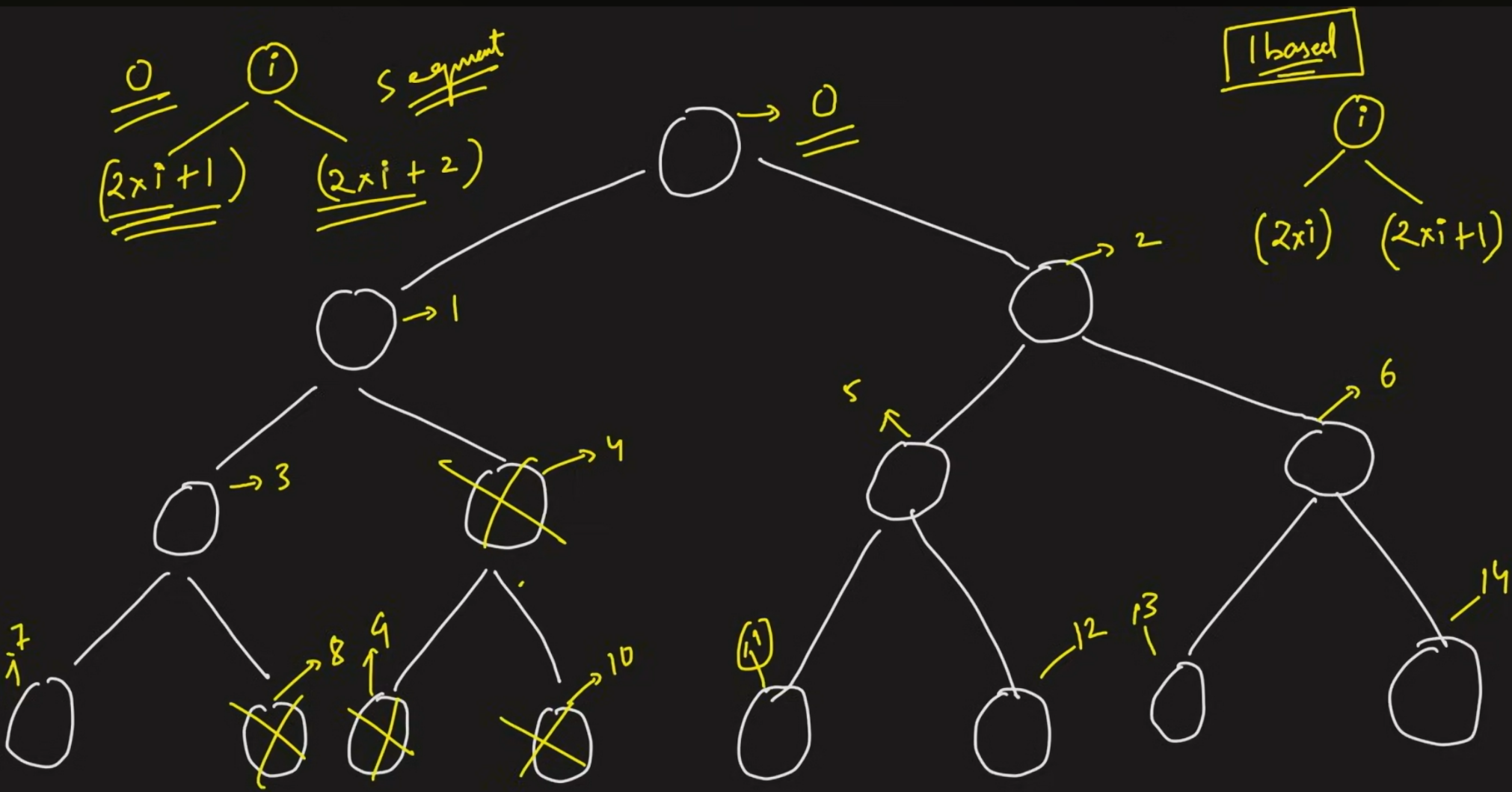
Width → Level

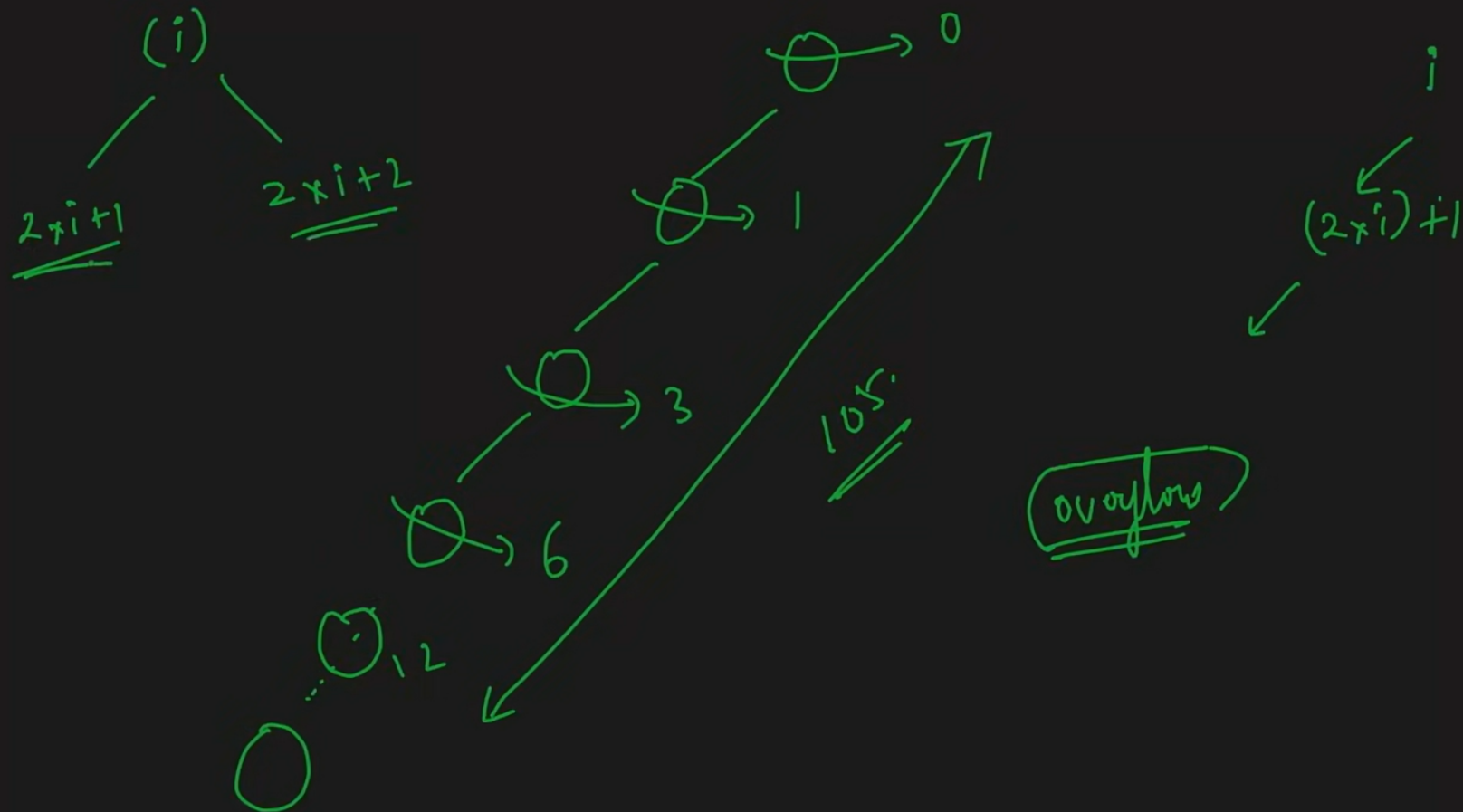
Level order Traversal

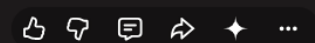
$$15 - 8 + 1 = 8$$

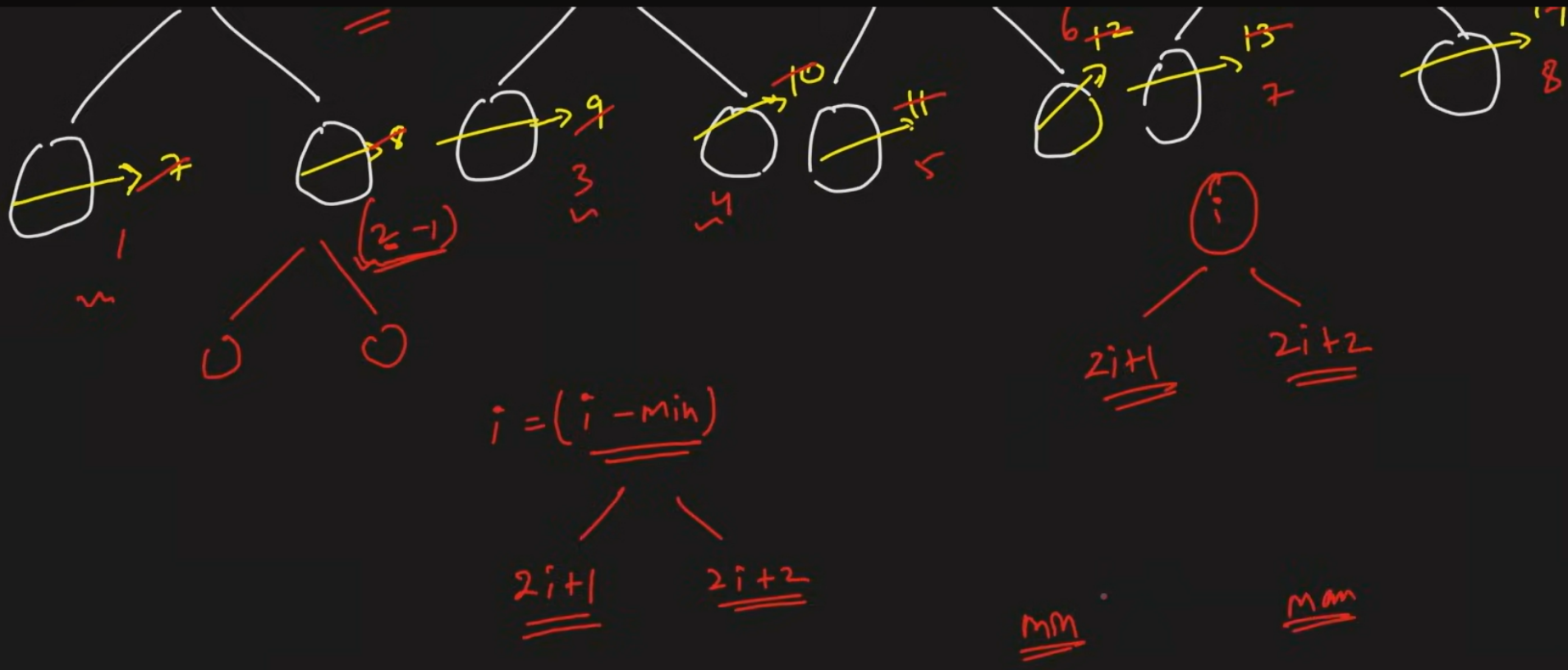
(last n - front node + 1)

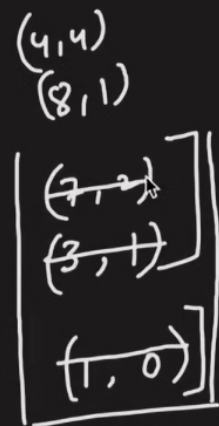
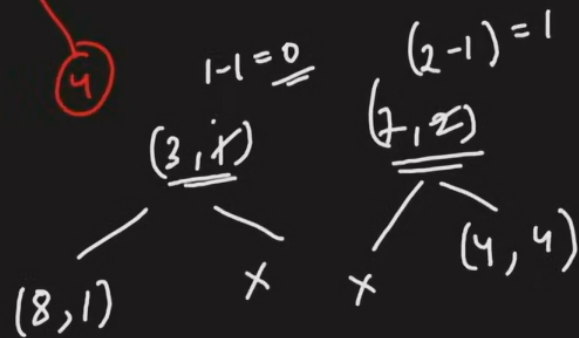
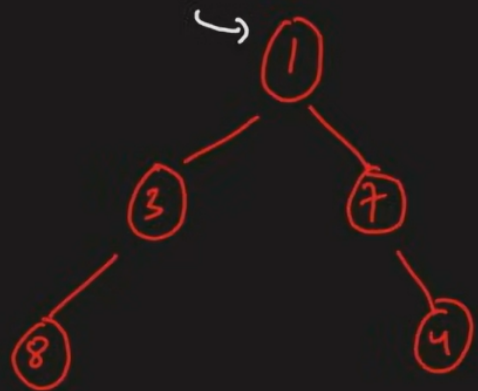












0

$width = 4^2$

$(last - first + 1)$

```

1  /**
2   * Definition for a binary tree node.
3   * struct TreeNode {
4   *     int val;
5   *     TreeNode *left;
6   *     TreeNode *right;
7   *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
8   *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
9   *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
10  * };
11  */
12  class Solution {
13  public:
14      int widthOfBinaryTree(TreeNode* root) {
15          if(!root)
16              return 0;
17          int ans = 0;
18          queue<pair<TreeNode*, int>> q;
19          q.push({root, 0});
20          while(!q.empty()){
21              int size = q.size();
22              int mmin = q.front().second; //to make the id starting from zero
23              int first, last;
24              for(int i=0; i<size; i++){
25                  int cur_id = q.front().second-mmin;
26                  TreeNode* node = q.front().first;
27                  q.pop();
28                  if(i==0) first = cur_id;
29                  if(i==size-1) last = cur_id;
30                  if(node->left)
31                      q.push({node->left, cur_id*2+1});
32                  if(node->right)
33                      q.push({node->right, cur_id*2+2});
34              }
35              ans = max(ans, last-first+1);
36          }
37          return ans;
38      }
39  };

```

C++



$$O(2-1) =$$